

What is Processor Design

Processor design is the process of defining and creating the internal architecture of a processor (CPU) so that it can execute instructions efficiently, handle data, interact with memory and I/O, and meet performance, power, cost and reliability constraints.

The purpose of processor design is to create an efficient, reliable, and cost-effective processor that meets system requirements such as performance, power efficiency, and scalability. It ensures that the processor implements its instruction set architecture (ISA) correctly while optimizing speed, area, and power consumption.

Design goals (performance, cost, power, reliability)

In processor design, one of the first tasks is to define **goals** which guide the architecture, micro-architecture and implementation. These typically include performance, cost, power and reliability.

1. Performance

Performance measures how fast the processor executes instructions and tasks.

Key metrics include:

Clock frequency: Speed of the CPU in GHz.

Instructions per cycle (IPC): Number of instructions executed per clock cycle.

Throughput: Total amount of work the processor can complete in a given time.

Latency: Time taken to complete a specific task.

Designers aim to **maximize performance** while satisfying other constraints like cost, power, and reliability.

2. Cost

Cost includes both:

Manufacturing cost: Such as die area, chip yield, and packaging.

Design cost: Engineering time and tool expenses.

Larger die area generally increases cost, so designers must consider **trade-offs between area, time, and power**.

3. Power

Power is critical, especially in **mobile, embedded, or large-scale systems**.

The goal is to **minimize power consumption and heat generation** without sacrificing performance.

Higher integration of circuits often **reduces power usage** and improves reliability.

4. Reliability

Reliability ensures that the processor operates correctly under:

Manufacturing variations

Temperature changes

Voltage fluctuations

Aging over time

Reliability is considered alongside performance, cost, and power as a major design goal.

5. Importance of These Goals

A processor that is extremely fast but **expensive, power-hungry, or unreliable** would be of limited use.

Designers must **balance trade-offs**, for example:

Increasing clock frequency may raise power consumption and cost.

Reducing die area may cause thermal issues or reduce performance.

Managing these trade-offs is a core part of processor design.

Processor design flow.

Processor Design Styles

Hennessy distinguishes three main implementation styles:

1. Single-Cycle Design

- Each instruction executes in **one clock cycle**.
- Simple but inefficient (clock must be long enough for the slowest instruction).
- Used for educational understanding, not real processors.

□ Multi-Cycle Design

- Instruction execution divided into **multiple cycles**.
- Shorter clock period; different instructions take different numbers of cycles.
- Improves resource utilization.

□ Pipelined Design

- **Most common in modern processors.**
- Overlaps instruction execution—like an assembly line.
- Increases throughput but introduces **hazards** (data, control, structural).

Instruction Set Architecture (ISA)

ISA is the interface between software and hardware, defining the instructions a processor supports, how they are encoded, how operands are accessed, and the addressing modes available. It ensures software can control hardware consistently.

Types of instructions (data movement, arithmetic, control)

Instructions are mainly of three types:

Data-movement: move data between memory and registers (e.g., LOAD, STORE).

Arithmetic/Logic: perform calculations or logical operations (e.g., ADD, SUB, AND).

Control: manage program flow (e.g., JUMP, BRANCH, CALL).

Instruction formats and addressing modes

Instruction formats define how bits in an instruction are organized, including opcode, operands, and addressing. Addressing modes specify how operands are accessed: immediate, register, direct, indirect, or indexed.

RISC vs CISC Architectures:

RISC (Reduced Instruction Set Computer) uses a small, simple, mostly fixed-length instruction set, which allows faster instruction execution, easier pipelining, and efficient use of registers. It emphasizes load-store architecture, meaning memory is accessed only through specific load and store instructions, while arithmetic and logic operations work on registers. CISC (Complex Instruction Set Computer), on the other hand, uses a larger, more complex, variable-length instruction set with many addressing modes, allowing some instructions to perform multiple operations in a single step. This can reduce program size but makes decoding and execution more complicated.

Datapath Design

Datapath design defines how data moves inside a processor to perform computations. The Arithmetic Logic Unit (ALU) performs arithmetic and logic operations, while registers temporarily store data and instructions, and the register file is a collection of these registers. Multiplexers (MUX) act as switches to select which input data goes to the ALU or other components based on control signals. Buses carry data between the ALU, registers, memory, and output devices. Together, these components control the flow of data inside the CPU efficiently.

Control Unit Design

The Control Unit (CU) manages and coordinates all parts of the processor, ensuring instructions are executed correctly. A Hardwired Control Unit uses fixed logic circuits to generate control signals for operations, making it fast but less flexible. A Microprogrammed Control Unit stores control signals as sequences of microinstructions in memory, which makes it easier to modify and support complex instructions. The CU sends control signals to components like the ALU, registers, and buses, and coordinates timing so that each operation happens in the correct order.

Pipelining

Pipelining in computer architecture is a technique used to increase the CPU's instruction throughput by overlapping the execution of multiple instructions. Instead of completing one instruction before starting the next, the processor divides execution into several stages that work in parallel. A typical instruction pipeline consists of five stages: **Instruction Fetch (IF)**, where the instruction is read from memory; **Instruction Decode (ID)**, where it is decoded and the operands are identified; **Execute (EX)**, where the Arithmetic Logic Unit (ALU) performs the required operation; **Memory Access (MEM)**, where data is read or written to memory; and **Write Back (WB)**, where the final result is stored in a register. This overlapping of operations allows the processor to complete one instruction per clock cycle in an ideal scenario, improving overall system performance.

However, pipelining also introduces certain problems known as **pipeline hazards**. There are three main types of hazards: **Structural hazards** occur when multiple instructions require the same hardware resource at the same time. **Data hazards** arise when an instruction depends on the result of a previous one that has not yet completed. **Control hazards** occur during branch or jump instructions when the processor is uncertain about which instruction to fetch next. These hazards prevent the smooth flow of instructions through the pipeline and can decrease performance if not managed properly

✖ Why Hazards Occur

In a **pipelined processor**, multiple instructions are executed *in parallel* — each in a different stage (Fetch, Decode, Execute, Memory, Write Back).

So, while one instruction is executing, another might be:

- Reading registers
- Writing to memory
- Branching to another address

This overlap causes **conflicts** between instructions — those conflicts are **hazards**.

⚙ Types of Hazards

According to Hennessy & Patterson, there are **three main types**:

❑ Structural Hazard (Resource Conflict)

Occurs when **two or more instructions need the same hardware resource** at the same time.

🧠 Example:

If the CPU has **one memory unit** for both instruction fetch and data access:

Cycle	Instruction 1	Instruction 2
1	Fetch	—
2	Execute	Fetch
3	Memory	Execute
4	—	Memory 🚫 (conflict!)

Solution:

- Separate **instruction** and **data memories** (Harvard architecture)
- Add **duplicate resources**

Data Hazard (Read-After-Write Conflict)

Occurs when an instruction **depends on the result** of a previous instruction that hasn't completed yet.

Example:

1. ADD R1, R2, R3 ; R1 = R2 + R3
2. SUB R4, R1, R5 ; needs the updated R1

But in pipelining, SUB may read R1 before ADD writes it → **incorrect value**.

Solutions:

- **Forwarding (bypassing):** Send result directly from ALU output to next instruction before it's written to register.
- **Stalling (pipeline bubble):** Delay the next instruction until data is ready.

Control Hazard (Branch Hazard)

Occurs due to **branch or jump instructions** — the processor doesn't know the next instruction's address until the branch is resolved.

Example:

```
BEQ R1, R2, Label  
ADD R3, R4, R5 ; fetched before knowing if branch is taken
```

If the branch is **taken**, the ADD was fetched **incorrectly**.

Solutions:

- **Branch prediction:** Guess the outcome of the branch.
- **Delayed branching:** Always execute the instruction after the branch.
- **Flushing the pipeline:** Cancel wrong instructions after branch resolution.

Memory System Design

Memory system design focuses on how data is stored and accessed efficiently to support the CPU. The **memory hierarchy** arranges storage types by speed and size from registers and cache (fastest, smallest) to main memory and secondary storage (slower, larger). This structure helps the processor access frequently used data quickly.

Cache memory is a small, fast memory close to the CPU that stores recently used data. It works in levels (L1, L2, L3) to reduce access time and improve performance. **Main memory (RAM)** stores active data and programs, while **virtual memory** uses part of the hard drive to extend usable memory when RAM is full.

The effectiveness of a memory system depends on **memory access time** (speed of reading/writing data) and **memory bandwidth** (amount of data transferred per second). A well-balanced design between cache, main, and virtual memory ensures high performance and efficient system operation

Input/Output (I/O) Design

Input/Output (I/O) design defines how a processor exchanges data with external devices like keyboards, printers, displays, and sensors. **I/O interfaces and controllers** act as middlemen between the CPU and these devices, managing control signals, timing, and data conversion so that devices with different speeds or data formats can work together efficiently.

Interrupts and Direct Memory Access (DMA) improve system performance. Interrupts let devices alert the CPU when they need attention, avoiding continuous checking. DMA allows data to move directly between memory and I/O devices without involving the CPU for every transfer, saving time and increasing speed.

Communication with peripherals ensures that input devices send data correctly to the system and output devices receive results properly. Together, these mechanisms allow fast, reliable, and well-coordinated data exchange between the processor, memory, and external hardware.

Bus and Interconnection Design

Bus and interconnection design defines how different parts of a computer like the CPU, memory, and I/O devices communicate with each other. The **system bus structure** has three main parts: the **address bus** (which carries memory addresses), the **data bus** (which transfers actual data), and the **control bus** (which carries control signals to manage operations).

Interconnect networks link multiple processors and memory units in complex systems, allowing data to move efficiently between components. These can be simple shared buses or advanced network topologies like crossbars and meshes used in multiprocessor systems.

Bus arbitration and data transfer ensure that when multiple devices try to use the bus at the same time, only one gains control at a time. Arbitration techniques like priority or round-robin decide which device accesses the bus first, maintaining order and preventing data conflicts.

Performance and Power Optimization

Performance and power optimization in processor design focuses on making the CPU faster while using less energy. **Clock speed** determines how many instructions a processor can execute per second, while **instruction throughput** measures how many instructions are completed in a given time both are key indicators of performance.

To save energy, processors use **power-saving techniques** such as clock gating (turning off unused parts of the chip), dynamic voltage scaling, and sleep modes that reduce power when full performance isn't needed.

Modern CPUs also improve efficiency through **parallelism and superscalar design**. Parallelism allows multiple instructions or tasks to run simultaneously, while superscalar processors have multiple execution units to complete more than one instruction per clock cycle. This combination increases speed without a big rise in power consumption.

Testing, Verification, and Implementation

After designing a processor, it must be thoroughly **tested and verified** to ensure it works correctly. **Simulation and verification** involve checking the processor design through software tools that model how the hardware will behave. This helps catch design errors before the chip is made.

Next, **hardware synthesis** converts the verified design (written in hardware description languages like Verilog or VHDL) into a physical circuit layout that can be manufactured. This stage translates logical design into real hardware components.

Finally, **fabrication and performance testing** involve producing the actual chip in a semiconductor foundry and then testing its real-world performance such as speed, power consumption, and reliability to make sure it meets design goals.